ANEXO I

```
#Lista de exercícios 1 - Mecânica dos Fluidos e Transferência de Calor computacional

# 1) a)

import numpy as np
import matplotlib.pyplot as plt
import time

for n in range(1):
    # Dados
    h = 10 # W/(m² * K)
    k = 1 # W/(m * K)
    dx = 0.25/(n+1) # m = dy
    Tinf = 300 # K
    Tsup = 500 # K
    Tchute = [480, 470, 440, 430, 400, 390, 370, 350]
    episilon = 0.1 # °C

    # Número de elementos da malha
    n_el = int((1/dx)**2)

    # Número de linhas e colunas da matriz N x M (considerando a simetria do problema)
    n_l = int((n_el)**(0.5) + 1)
    n_col = int((n_l+1)/2)

    # Vetor de temperaturas
    temps = []
    for i in range((n_l-1)*(n_col-1)):
        temps.append([Tchute[i]])
```

```
# Matriz inicial
m = np.zeros((n_l,n_col))

# Preenche os valores das bordas
for i in range(m.shape[0]):
    for j in range(m.shape[1]):
        if i==0 or j == 0:
            m[i,j] = Tsup

# Preenche os valores iniciais das temperaturas
c = 0
for i in range(m.shape[0]-1):
    for j in range(m.shape[1]-1):
            m[i+1,j+1] = temps[c][-1]
            c += 1
cut = True

def chk_erro(vec, e_cut):
    chk = []
    for i in range (len(vec)):
        chk.append(abs(vec[i][-1] - vec[i][-2]) < e_cut)
    if all(chk):
        return False
    else:
        return True
inicio = time.perf_counter()
it = 0
while cut:
    c = 0
    for i in range(m.shape[0]):
        for j in range(m.shape[1]):
                # Cálculo das temperaturas internas simétricas
                if i != 0 and i != m.shape[0]-1 and j != 0 and j != m.shape[1] - 1:
                    m[i,j] = (m[i,j-1] + m[i-1,j] + m[i,j+1] + m[i+1,j])/(4)
```

```
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo das temperaturas sobre a adiábata submetidas à condução
            if i != 0 and i != m.shape[0]-1 and j == m.shape[1] - 1:
                m[i,j] = (2 * m[i,j-1] + m[i-1,j] + m[i+1,j])/(4)
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo das temperaturas simétricas submetidas à condução + convecção
            if j != 0 and j != m.shape[1] - 1 and i == m.shape[0]-1:
                m[i,j] = (m[i,j-1] + m[i,j+1] + 2 * m[i-1,j] + 2 * h * dx / k * Tinf)/(4 + 2 * h * dx / k)
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo da temperatura sobre a adiábata submetida à condução + convecção
            if j == m.shape[1] - 1 and i == m.shape[0] - 1:
                m[i,j] = (m[i-1,j] + m[i,j-1] + h * dx / k * Tinf)/(2 + h * dx / k)
                c += 1
                temps[c-1].append(float(m[i,j]))
    it += 1
    cut = chk_erro(temps, episilon)
fim = time.perf_counter()

# Espelhando a matriz simétrica
# Matriz de Zeros com as dimensões completas
mf = np.zeros((n_l,n_l))
# Preenche as colunas da matriz completa com os valores calculados
mf[:,:n_col] = m
# Para preencher as colunas faltantes (simetria)
mf[:,(n_col):] = m[:,:-1][:,-1::-1]

# Matriz com as temperaturas obtidas
print("Matriz com as temperaturas obtidas")
print("\n")
print(mf)
print("\n")
```

```
    for i in range(8):
        if i % 2 == 0:
            print(f"T{(i+1)}: {m[int(((i+1)+1)/2*(n+1)),(n+1)]:.2f}")
        if i % 2 != 0:
            print(f"T{(i+1)}: {m[int((i+1)/2*(n+1)),(n+1)*2]:.2f}")

    print("\n")

    print(f"Tempo total de execução do método: {fim - inicio:.6f} segundos")

    print("\n")

    print(f"Número de iterações necessárias: {it-1}\n\n")

# 1) c) Análise com $\varepsilon = 0,01\,\circ\mathrm{C}$

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import time
plotvec = [[],[]] # Vetor com as temperaturas $T_7$ e $T_8$, que serão usadas na análise de convergência de malha.
vn_el = [] # Vetor com os números de elementos das malhas
tit = [] # Vetor com os tempos de iteração
n_int = [0]
# Valores para dividir o comprimento incial da malha por múltiplos de 5 até 50
for i in range(5):
    n_int.append(5*(i+1) - 1)

for n in n_int:
    # Dados
    h = 10 # W/(m² * K)
    k = 1 # W/(m * K)
    dx = 0.25/(n+1) # m = dy
```

```
Tinf = 300 # K
Tsup = 500 # K
Tchute = 480 # K
episilon = 0.01 # °C

# Número de elementos da malha
n_el = int((1/dx)**2)
print(f"Número de elementos: {n_el} - malha de {dx:.5f} mm")
# Número de linhas e colunas da matriz N x M (considerando a simetria do problema)
n_l = int((n_el)**(0.5) + 1)
n_col = int((n_l+1)/2)

# Vetor de temperaturas
temps = []
for i in range((n_l-1)*(n_col-1)):
    temps.append([Tchute])

# Matriz inicial
m = np.zeros((n_l,n_col))

# Preenche os valores das bordas
for i in range(m.shape[0]):
    for j in range(m.shape[1]):
        if i==0 or j == 0:
            m[i,j] = Tsup

# Preenche os valores iniciais das temperaturas
c = 0
for i in range(m.shape[0]-1):
    for j in range(m.shape[1]-1):
            m[i+1,j+1] = temps[c][-1]
            c += 1
cut = True
```

```
def chk_erro(vec, e_cut):
    chk = []
    for i in range (len(vec)):
        chk.append(abs(vec[i][-1] - vec[i][-2]) < e_cut)
    if all(chk):
        return False
    else:
        return True
inicio = time.perf_counter() # Exercício 1) b)
it = 0
while cut:
    c = 0
    for i in range(m.shape[0]):
        for j in range(m.shape[1]):
            # Cálculo das temperaturas internas simétricas
            if i != 0 and i != m.shape[0]-1 and j != 0 and j != m.shape[1] - 1:
                m[i,j] = (m[i,j-1] + m[i-1,j] + m[i,j+1] + m[i+1,j])/(4)
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo das temperaturas sobre a adiábata submetidas à condução
            if i != 0 and i != m.shape[0]-1 and j == m.shape[1] - 1:
                m[i,j] = (2 * m[i,j-1] + m[i-1,j] + m[i+1,j])/(4)
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo das temperaturas simétricas submetidas à condução + convecção
            if j != 0 and j != m.shape[1] - 1 and i == m.shape[0]-1:
                m[i,j] = (m[i,j-1] + m[i,j+1] + 2 * m[i-1,j] + 2 * h * dx / k * Tinf)/(4 + 2 * h * dx / k)
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo da temperatura sobre a adiábata submetida à condução + convecção
            if j == m.shape[1] - 1 and i == m.shape[0] - 1:
                m[i,j] = (m[i-1,j] + m[i,j-1] + h * dx / k * Tinf)/(2 + h * dx / k)
                c += 1
                temps[c-1].append(float(m[i,j]))
```

```
    it += 1
    cut = chk_erro(temps, episilon)
fim = time.perf_counter()
texec = round(fim - inicio,6)
tit.append(texec) # Guarda os tempos de execução
# Espelhando a matriz simétrica
# Matriz de Zeros com as dimensões completas
mf = np.zeros((n_l,n_l))
# Preenche as colunas da matriz completa com os valores calculados
mf[:,:n_col] = m
# Para preencher as colunas faltantes (simetria)
mf[:,(n_col):] = m[:,:-1][:,-1::-1]

# Matriz com as temperaturas obtidas
print("Matriz com as temperaturas obtidas")
print("\n")
print(mf)
print("\n")

print("\n")

for i in range(8):
    if i % 2 == 0:
        t = m[int((i+2)/2*(n+1)),(n+1)]
        print(f"T{(i+1)}: {t:.2f}")
        # Guardando a temperatura T7 para estudo de convergência de malha.
        if i == 6:
            plotvec[0].append(float(t))
    if i % 2 != 0:
        t = m[int((i+1)/2*(n+1)),(n+1)*2]
        print(f"T{(i+1)}: {t:.2f}")
        #Guardando a temperatura T8 para estudo de convergência de malha.
        if i == 7:
            plotvec[1].append(float(t))
```

```
    print("\n")

    print(f"Tempo total de execução do método: {texec} segundos")

    print("\n")

    print(f"Número de iterações necessárias: {it}\n\n")

    # Guardando o número de elementos desse estudo
    vn_el.append(n_el)

# Gráficos

plt.rcParams.update({
    "text.usetex": True,
    "font.family": "serif",
    "axes.labelsize": 14,
    "axes.titlesize": 16,
    "legend.fontsize": 12,
    "xtick.labelsize": 12,
    "ytick.labelsize": 12,
})

# Refino de malha - Temperaturas $T_7$ e $T_8$
plt.figure(figsize=(10,6))
plt.xlabel("Número de elementos da malha")
plt.ylabel("Temperatura " + r"($^\circ$C)")
lb = ["T7", "T8"]
for i in range(len(plotvec)):
    plt.plot(vn_el, plotvec[i], label = lb[i], marker = "o")
plt.xticks(vn_el, rotation=45)
plt.grid(True)
plt.legend()
```

```
plt.show()


# Refino de malha - Diferenças das temperaturas na nova malha e na imediatamente anterior
vn_el_t = vn_el[1:]
k = [[],[]]
for i in range(len(plotvec)):
    for j in range(1,len(plotvec[i])):
        k[i].append(abs(plotvec[i][j]-plotvec[i][j-1]))
plt.figure(figsize=(10,6))
plt.xlabel("Número de elementos da malha")
plt.ylabel("Diferença entre as temperaturas"+ r"($^\circ$C)")
for i in range(len(k)):
    plt.plot(vn_el_t, k[i], label = lb[i], marker = "o")
plt.xticks(vn_el_t)
plt.grid(True)
plt.legend()
plt.show()


# Refino de malha - Tempo das iterações
plt.figure(figsize=(10,6))
plt.xlabel("Número de elementos da malha")
plt.ylabel("Tempo de execução (s)")
plt.plot(vn_el, tit, marker = "o")
plt.xticks(vn_el, rotation=45)
plt.grid(True)
plt.show()


# 1) c) Análise com $\varepsilon = 10^{-8}\,\circ\mathrm{C}$


import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import time
plotvec = [[],[]] # Vetor com as temperaturas $T_7$ e $T_8$, que serão usadas na análise de convergência de malha.
```

```
vn_el = [] # Vetor com os números de elementos das malhas
tit = [] # Vetor com os tempos de iteração
n_int = [0]
# Valores para dividir o comprimento incial da malha por múltiplos de 5 até 50
for i in range(5):
    n_int.append(5*(i+1) - 1)

for n in n_int:
    # Dados
    h = 10 # W/(m² * K)
    k = 1 # W/(m * K)
    dx = 0.25/(n+1) # m = dy
    Tinf = 300 # K
    Tsup = 500 # K
    Tchute = 480 # K
    episilon = 10**(-8) # °C

    # Número de elementos da malha
    n_el = int((1/dx)**2)
    print(f"Número de elementos: {n_el} - malha de {dx:.5f} mm")
    # Número de linhas e colunas da matriz N x M (considerando a simetria do problema)
    n_l = int((n_el)**(0.5) + 1)
    n_col = int((n_l+1)/2)

    # Vetor de temperaturas
    temps = []
    for i in range((n_l-1)*(n_col-1)):
        temps.append([Tchute])

    # Matriz inicial
    m = np.zeros((n_l,n_col))

    # Preenche os valores das bordas
    for i in range(m.shape[0]):
```

```
    for j in range(m.shape[1]):
        if i==0 or j == 0:
            m[i,j] = Tsup

# Preenche os valores iniciais das temperaturas
c = 0
for i in range(m.shape[0]-1):
    for j in range(m.shape[1]-1):
            m[i+1,j+1] = temps[c][-1]
            c += 1
cut = True

def chk_erro(vec, e_cut):
    chk = []
    for i in range (len(vec)):
        chk.append(abs(vec[i][-1] - vec[i][-2]) < e_cut)
    if all(chk):
        return False
    else:
        return True
inicio = time.perf_counter() # Exercício 1) b)
it = 0
while cut:
    c = 0
    for i in range(m.shape[0]):
        for j in range(m.shape[1]):
                # Cálculo das temperaturas internas simétricas
                if i != 0 and i != m.shape[0]-1 and j != 0 and j != m.shape[1] - 1:
                    m[i,j] = (m[i,j-1] + m[i-1,j] + m[i,j+1] + m[i+1,j])/(4)
                    c += 1
                    temps[c-1].append(float(m[i,j]))
                # Cálculo das temperaturas sobre a adiábata submetidas à condução
                if i != 0 and i != m.shape[0]-1 and j == m.shape[1] - 1:
                    m[i,j] = (2 * m[i,j-1] + m[i-1,j] + m[i+1,j])/(4)
```

```
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo das temperaturas simétricas submetidas à condução + convecção
            if j != 0 and j != m.shape[1] - 1 and i == m.shape[0]-1:
                m[i,j] = (m[i,j-1] + m[i,j+1] + 2 * m[i-1,j] + 2 * h * dx / k * Tinf)/(4 + 2 * h * dx / k)
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo da temperatura sobre a adiábata submetida à condução + convecção
            if j == m.shape[1] - 1 and i == m.shape[0] - 1:
                m[i,j] = (m[i-1,j] + m[i,j-1] + h * dx / k * Tinf)/(2 + h * dx / k)
                c += 1
                temps[c-1].append(float(m[i,j]))
    it += 1
    cut = chk_erro(temps, episilon)
fim = time.perf_counter()
texec = round(fim - inicio,6)
tit.append(texec) # Guarda os tempos de execução
# Espelhando a matriz simétrica
# Matriz de Zeros com as dimensões completas
mf = np.zeros((n_l,n_l))
# Preenche as colunas da matriz completa com os valores calculados
mf[:,:n_col] = m
# Para preencher as colunas faltantes (simetria)
mf[:,(n_col):] = m[:,:-1][:,-1::-1]

# Matriz com as temperaturas obtidas
print("Matriz com as temperaturas obtidas")
print("\n")
print(mf)
print("\n")

print("\n")

for i in range(8):
```

```
            if i % 2 == 0:
                t = m[int((i+2)/2*(n+1)),(n+1)]
                print(f"T{(i+1)}: {t:.2f}")
                # Guardando a temperatura T7 para estudo de convergência de malha.
                if i == 6:
                    plotvec[0].append(float(t))
            if i % 2 != 0:
                t = m[int((i+1)/2*(n+1)),(n+1)*2]
                print(f"T{(i+1)}: {t:.2f}")
                #Guardando a temperatura T8 para estudo de convergência de malha.
                if i == 7:
                    plotvec[1].append(float(t))

        print("\n")

        print(f"Tempo total de execução do método: {texec} segundos")

        print("\n")

        print(f"Número de iterações necessárias: {it}\n\n")

        # Guardando o número de elementos desse estudo
        vn_el.append(n_el)

    # Gráficos

    plt.rcParams.update({
        "text.usetex": True,
        "font.family": "serif",
        "axes.labelsize": 14,
        "axes.titlesize": 16,
        "legend.fontsize": 12,
        "xtick.labelsize": 12,
        "ytick.labelsize": 12,
```

```
})

# Refino de malha - Temperaturas $T_7$ e $T_8$
plt.figure(figsize=(10,6))
plt.xlabel("Número de elementos da malha")
plt.ylabel("Temperatura " + r"($^\circ$C)")
lb = ["T7", "T8"]
for i in range(len(plotvec)):
    plt.plot(vn_el, plotvec[i], label = lb[i], marker = "o")
plt.xticks(vn_el, rotation=45)
plt.grid(True)
plt.legend()
plt.show()

# Refino de malha - Diferenças das temperaturas na nova malha e na imediatamente anterior
vn_el_t = vn_el[1:]
k = [[],[]]
for i in range(len(plotvec)):
    for j in range(1,len(plotvec[i])):
        k[i].append(abs(plotvec[i][j]-plotvec[i][j-1]))
plt.figure(figsize=(10,6))
plt.xlabel("Número de elementos da malha")
plt.ylabel("Diferença entre as temperaturas"+ r"($^\circ$C)")
for i in range(len(k)):
    plt.plot(vn_el_t, k[i], label = lb[i], marker = "o")
plt.xticks(vn_el_t)
plt.grid(True)
plt.legend()
plt.show()

# Refino de malha - Tempo das iterações
plt.figure(figsize=(10,6))
plt.xlabel("Número de elementos da malha")
plt.ylabel("Tempo de execução (s)")
```

```
plt.plot(vn_el, tit, marker = "o")

plt.xticks(vn_el, rotation=45)

plt.grid(True)

plt.show()


#d)


import numpy as np

import matplotlib.pyplot as plt

from matplotlib.colors import LogNorm

import time

plotvec = [[],[]] # Vetor com as temperaturas $T_7$ e $T_8$, que serão usadas na análise de convergência de malha.

plotdif = [] # Vetor com as diferenças das temperaturas $T_7$ e $T_8$

v_eps = [] # Vetor com os valores de epsilon

tit = [] # Vetor com os tempos de iteração

for n in range(7,-1,-1): # De 8 até 0

    # Dados

    h = 10 # W/(m² * K)

    k = 1 # W/(m * K)

    dx = 0.25/10 # m = dy - Tamanho de malha equivalente a 1600 elementos

    Tinf = 300 # K

    Tsup = 500 # K

    Tchute = 480 # K

    episilon = 10**(-(n+1)) # °C De  $10^{-8}$ até $0,01$

    print(f"Número da iteração: {8-n} - malha de {dx:.5f} mm")


    # Número de elementos da malha

    n_el = int((1/dx)**2)


    # Número de linhas e colunas da matriz N x M (considerando a simetria do problema)

    n_l = int((n_el)**(0.5) + 1)

    n_col = int((n_l+1)/2)


    # Vetor de temperaturas
```

```
temps = []
for i in range((n_l-1)*(n_col-1)):
    temps.append([Tchute])

# Matriz inicial
m = np.zeros((n_l,n_col))

# Preenche os valores das bordas
for i in range(m.shape[0]):
    for j in range(m.shape[1]):
        if i==0 or j == 0:
            m[i,j] = Tsup

# Preenche os valores iniciais das temperaturas
c = 0
for i in range(m.shape[0]-1):
    for j in range(m.shape[1]-1):
            m[i+1,j+1] = temps[c][-1]
            c += 1
cut = True

def chk_erro(vec, e_cut):
    chk = []
    for i in range (len(vec)):
        chk.append(abs(vec[i][-1] - vec[i][-2]) < e_cut)
    if all(chk):
        return False
    else:
        return True
inicio = time.perf_counter()
it = 0
while cut:
    c = 0
    for i in range(m.shape[0]):
```

```
        for j in range(m.shape[1]):
            # Cálculo das temperaturas internas simétricas
            if i != 0 and i != m.shape[0]-1 and j != 0 and j != m.shape[1] - 1:
                m[i,j] = (m[i,j-1] + m[i-1,j] + m[i,j+1] + m[i+1,j])/(4)
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo das temperaturas sobre a adiábata submetidas à condução
            if i != 0 and i != m.shape[0]-1 and j == m.shape[1] - 1:
                m[i,j] = (2 * m[i,j-1] + m[i-1,j] + m[i+1,j])/(4)
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo das temperaturas simétricas submetidas à condução + convecção
            if j != 0 and j != m.shape[1] - 1 and i == m.shape[0]-1:
                m[i,j] = (m[i,j-1] + m[i,j+1] + 2 * m[i-1,j] + 2 * h * dx / k * Tinf)/(4 + 2 * h * dx / k)
                c += 1
                temps[c-1].append(float(m[i,j]))
            # Cálculo da temperatura sobre a adiábata submetida à condução + convecção
            if j == m.shape[1] - 1 and i == m.shape[0] - 1:
                m[i,j] = (m[i-1,j] + m[i,j-1] + h * dx / k * Tinf)/(2 + h * dx / k)
                c += 1
                temps[c-1].append(float(m[i,j]))
    it += 1
    cut = chk_erro(temps, episilon)
fim = time.perf_counter()
texec = round(fim - inicio,6)
tit.append(texec) # Guarda os tempos de execução
# Espelhando a matriz simétrica
# Matriz de Zeros com as dimensões completas
mf = np.zeros((n_l,n_l))
# Preenche as colunas da matriz completa com os valores calculados
mf[:,:n_col] = m
# Para preencher as colunas faltantes (simetria)
mf[:,(n_col):] = m[:,:-1][:,-1::-1]
```

```
# Matriz com as temperaturas obtidas
print("Matriz com as temperaturas obtidas")
print("\n")
print(mf)
print("\n")

print("\n")

# Printando os pontos $T_1$ a $T_8$
for i in range(8):
    if i % 2 == 0:
        t = m[int((i+2)/2*10),10]
        print(f"T{(i+1)}: {t:.2f}")
        # Guardando a temperatura T7 para estudo de convergência de malha.
        if i == 6:
            plotvec[0].append(float(t))
    if i % 2 != 0:
        t = m[int((i+1)/2*10),10*2]
        print(f"T{(i+1)}: {t:.2f}")
        #Guardando a temperatura T8 para estudo de convergência de malha.
        if i == 7:
            plotvec[1].append(float(t))

print("\n")

print(f"Tempo total de execução do método: {texec} segundos")

print("\n")

print(f"Número de iterações necessárias: {it}\n\n")

# Guardando o número de elementos desse estudo
v_eps.append(episilon)
```

```
# Gráficos


plt.rcParams.update({
    "text.usetex": True,
    "font.family": "serif",
    "axes.labelsize": 14,
    "axes.titlesize": 16,
    "legend.fontsize": 12,
    "xtick.labelsize": 12,
    "ytick.labelsize": 12,
})


# Estudo do critério  de convergênca - Temperaturas $T_7$ e $T_8$
plt.figure(figsize=(10,6))
plt.xlabel("Critério de convergência " + r"($^\circ$C)")
plt.ylabel("Temperatura " + r"($^\circ$C)")
lb = ["T7", "T8"]
for i in range(len(plotvec)):
    plt.plot(v_eps, plotvec[i], label = lb[i], marker = "o")
plt.xscale('log')
plt.xticks(v_eps)
plt.grid(True)
plt.legend()
plt.show()


# Estudo do critério de convergência - Diferenças das temperaturas encontradas e a referência
v_eps_t = v_eps[1:] # Retirada do primeiro elemento pois ele é a referência
p = [[],[]]
for i in range(len(plotvec)):
    for j in range(1,len(plotvec[i])):
        p[i].append(abs(plotvec[i][0]-plotvec[i][j])) # Diferença entre a temperatura no $\varepsilon$ atual e a referência
plt.figure(figsize=(10,6))
plt.xlabel("Critério de convergência " + r"($^\circ$C)")
plt.ylabel("Erro " + r"($|T(10^{-8}) - T(\varepsilon)|)\,(^\circ\mathrm{C}$)")
```

```
for i in range(len(p)):
    plt.plot(v_eps_t, p[i], label = lb[i], marker = "o")
plt.xscale('log')
plt.yscale('log')
plt.xticks(v_eps_t)
plt.grid(True)
plt.legend()
plt.show()

# Razão entre o erro e o $\varepsilon para $T_7$
for i in range(len(k[0])):
    print(k[0][i]/v_eps_t[i])

# Razão entre o erro e o $\varepsilon para $T_8$
for i in range(len(k[0])):
    print(k[0][i]/v_eps_t[i])

# Estudo do critério de convergência - Tempo das iterações
plt.figure(figsize=(10,6))
plt.xlabel("Critério de convergência " + r"($^\circ$C)")
plt.ylabel("Tempo de execução (s)")
plt.plot(v_eps, tit, marker = "o")
plt.xscale('log')
plt.xticks(v_eps)
plt.grid(True)
plt.show()

# e)

# Distribuição de temperaturas 3-D
fig = plt.figure(figsize=(10,6))
ax = fig.add_subplot(projection='3d')
X = np.arange(0,1+dx,dx)
Y = np.arange(0,1+dx,dx)
```

```
X,Y = np.meshgrid(X,Y)
Z = mf
ax.view_init(elev=30,azim=45)
ax.plot_surface(X,Y,Z,rstride=1,cstride=1,cmap='viridis')
ax.set_xlabel("x " + r"($\mathrm{m}$)")
ax.set_ylabel("y " + r"($\mathrm{m}$)")
ax.text2D(
    -0.08, 0.5,
    "Temperatura " + r"($^\circ\mathrm{C}$)",
    transform=ax.transAxes,
    rotation=90,
    va='center'
)
plt.show()

# f)

# Gráfico campo vetorial do fluxo de calor

# Fluxo (com a correção da face convectiva)
dTdy, dTdx = np.gradient(mf, dx)
qx = -k * dTdx
qy = -k * dTdy
qy[-1, :] = h * (mf[-1, :] - Tinf)

# Malha de coordenadas
x = np.arange(0, 1 + dx/2, dx)
y = np.arange(0, 1 + dx/2, dx)
X, Y = np.meshgrid(x, y)

# Subamostragem
s = 4
Xs, Ys, Zs = X[::s, ::s], Y[::s, ::s], mf[::s, ::s]
qxs, qys = qx[::s, ::s], qy[::s, ::s]
```

```
dTdxs, dTdys = dTdx[::s, ::s], dTdy[::s, ::s]

# Componente vertical: projeção sobre a superfície
qzs = qxs * dTdxs + qys * dTdys

# Normalização (comprimento uniforme)
mag = np.sqrt(qxs**2 + qys**2)
norm3 = np.sqrt(qxs**2 + qys**2 + qzs**2)
norm3[norm3 == 0] = 1
u, v, w = qxs/norm3, qys/norm3, (qzs/norm3)

# Cor pela magnitude do fluxo horizontal
cmap = plt.get_cmap("Greys")
cores = cmap(mag.ravel() / mag.max())

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(projection='3d')
ax.plot_surface(X, Y, mf, cmap='viridis', alpha=0.25,
                rstride=1, cstride=1, linewidth=0)
ax.quiver(Xs, Ys, Zs, u, v, w, length=0.5, normalize=False,
          colors=cores, linewidth=2, arrow_length_ratio = 0.5)
ax.set_xlabel(r"$x$ (m)")
ax.set_ylabel(r"$y$ (m)")
ax.set_zlabel(r"Temperatura (K)")
ax.view_init(elev=25, azim=45)

sm = plt.cm.ScalarMappable(cmap=cmap,
        norm=plt.Normalize(vmin=0, vmax=mag.max()))
fig.colorbar(sm, ax=ax, shrink=0.6,
             label=r"$|\vec{q}"|$ (W/m$^2$)")
plt.show()
```